

GUIA COMPLETO • PT-BR

> kit-ia-dev

Kit IA Dev

Sua IA Vai Conhecer Seu Projeto em 5 Minutos

O sistema que elimina o “deixa eu explicar tudo de novo” toda vez que você abre uma sessão de IA — configurado pela própria IA, não por você.

Índice

01 INTRODUÇÃO

O que você comprou e por que isso funciona · Como usar este guia

02 FUNDAMENTOS — O PORQUÊ TÉCNICO

O problema real que isso resolve · A solução: carregamento sob demanda · Por que uma entrevista, e não um formulário · O que são “skills”, tecnicamente · Por que um hook, e não só uma instrução

03 WORKFLOW 1 — SETUP EM 5 MINUTOS

Passo 1: Onde colocar os arquivos · Passo 2: O comando que inicia tudo · Passo 3: A entrevista · Passo 4: Revisando o resultado · Passo 5: Commit e pronto

04 WORKFLOW 2 — AS 10 SKILLS NO DIA A DIA

Como uma skill é ativada · As 10 skills, uma por uma, com exemplo real · Quando pedir explicitamente

05 EXEMPLOS COMPLETOS — ANTES E DEPOIS

Code Review, Debug Assistant, Security Audit e Feature Planner em ação

06 WORKFLOW 3 — MÚLTIPLAS FERRAMENTAS

Por que isso importa · Claude Code, Cursor, Copilot, e outras · Adicionando uma ferramenta depois

07 WORKFLOW 4 — MANTENDO O SISTEMA VIVO

O sistema aprende com você · O hook de validação · Quando e como atualizar

08 CASOS DE USO POR TIPO DE PROJETO

Projeto novo, existente, em equipe, API, dados sensíveis, monorepo

09 TROUBLESHOOTING

Organizado por sintoma

10 PERGUNTAS FREQUENTES

Dúvidas sobre uso, escopo e limites do sistema

Todos os arquivos do kit, em uma página · Comandos que você vai usar mais

Introdução

O QUE VOCÊ COMPROU E POR QUE ISSO FUNCIONA

Toda sessão nova de IA — Claude, Cursor, ou qualquer outra — começa do zero. Ela não sabe qual é a stack do seu projeto, quais convenções você segue, o que nunca pode ser feito, ou quais regras de negócio o sistema precisa respeitar. Você explica tudo de novo, toda vez. Isso consome uns 20 minutos por sessão — e se você abre 3 ou 4 sessões por dia, são mais de uma hora do seu tempo, todo santo dia, só repetindo contexto que a IA deveria simplesmente já saber.

É exatamente essa dor que a pesquisa mais recente do setor confirma: segundo o Stack Overflow Developer Survey (49 mil desenvolvedores, 177 países), 84% já usam ou planejam usar ferramentas de IA no trabalho, mas apenas 33% confiam na precisão do que essas ferramentas produzem — e 46% desconfiam ativamente. O motivo mais citado, por 66% dos desenvolvedores, é receber “soluções quase certas, mas não exatamente” — código que parece funcionar, mas ignora uma convenção ou regra que nunca foi passada como contexto.

Este kit resolve isso de um jeito específico, e rápido: em 5 minutos, você tem um sistema completo rodando no seu projeto — não um arquivo estático que você preenche manualmente, mas um sistema que a própria IA configura por você, através de uma entrevista guiada, pesquisando as práticas mais atuais para o seu stack antes de escrever qualquer recomendação. Depois de configurado, ele se mantém sozinho — existe uma instrução dentro do sistema que faz a IA sugerir atualizações sempre que percebe algo importante que deveria virar regra permanente.

Isso é diferente de um curso ou de um pacote de prompts genéricos. Não tem aula, não tem teoria — é infraestrutura que você instala uma vez e sua IA passa a conhecer o seu projeto para sempre, sem depender de você lembrar de explicar tudo outra vez.

O QUE ESTÁ INCLUÍDO

ITEM	O QUE É
CLAUDE.md + sistema de arquivos	O núcleo do kit — configuração que a IA preenche entrevistando você em minutos
10 Skills	Procedimentos especializados que ativam automaticamente conforme a tarefa
Este guia	Passo a passo direto ao ponto, em português
30 prompts contextualizados	Para quem usa a IA pela web, sem instalar nada
Dashboard HTML	Referência visual rápida de tudo que você tem disponível
Checklist de 1 página	Os 10 erros mais comuns que fazem a IA gerar código ruim

COMO USAR ESTE GUIA

Este guia tem quatro workflows principais. Se você só quer instalar e começar a usar agora, vá direto pro Workflow 1 — em 5 minutos você já está rodando. Os outros três você consulta quando a situação pedir, não antes. O Troubleshooting no final está organizado por sintoma (o que está acontecendo de errado), não por funcionalidade — assim você acha a resposta procurando pelo problema, não adivinhando qual seção cobre ele.

Uma recomendação direta: não leia o guia inteiro antes de começar. Abra o Workflow 1 agora, no seu próprio projeto, com o guia ao lado. Em menos tempo do que levaria pra terminar de ler esta introdução, seu sistema já pode estar rodando.

O Porquê Técnico

Esta seção é opcional na primeira leitura — se você só quer instalar e usar, pode pular direto para o Workflow 1. Mas se você quer entender por que o sistema foi desenhado do jeito que foi (e não como um único arquivo gigante, por exemplo), vale a pena ler antes de começar.

O PROBLEMA REAL QUE ISSO RESOLVE

Uma IA de código não tem memória entre sessões. Cada vez que você abre uma conversa nova, ela começa do zero — sem saber o que é o seu projeto, como ele é estruturado, ou o que você já decidiu sobre ele. A solução óbvia parece simples: escrever tudo isso em um arquivo e pedir para a IA ler esse arquivo no início de cada sessão.

O problema é que essa solução simples tem um limite real. Modelos de IA processam um número finito de “tokens” de contexto por sessão, e, mais importante: quanto mais informação irrelevante está presente naquele momento, pior fica a precisão do modelo nas informações que realmente importam. Isso não é uma limitação de um modelo específico — é um padrão observado de forma consistente em múltiplos modelos testados: depois de um certo volume de contexto acumulado, a precisão cai, mesmo que o modelo “veja” tecnicamente toda a informação.

Na prática, isso significa: se você colocar tudo — regras de negócio, convenções de código, políticas de segurança, decisões de arquitetura, guia de produtividade — dentro de um único arquivo gigante, a IA começa a “esquecer” partes dele, mesmo que estejam lá escritas. Um arquivo de 2.000 linhas não é 10x mais útil que um de 200 linhas — muitas vezes é pior, porque a informação que importa fica diluída.

A SOLUÇÃO: CARREGAMENTO SOB DEMANDA

O nome técnico para o padrão que este kit usa é *progressive disclosure* (divulgação progressiva). A ideia central: nem toda informação precisa estar presente o tempo todo. Só o que é verdadeiramente universal — usado em toda e qualquer tarefa — fica no arquivo principal (CLAUDE.md). O resto fica em arquivos separados, que a IA só lê quando a tarefa realmente pede aquele contexto específico.

É por isso que o kit tem essa estrutura:

- `CLAUDE.md` — o que é necessário sempre: contexto do projeto, comandos, estrutura de pastas, e “ponteiros” dizendo onde encontrar o resto.
- `agent_docs/` — o que é necessário às vezes: regras de negócio só entram em jogo quando a tarefa toca uma área sensível; segurança só quando o código lida com dados sensíveis; produtividade só quando faz sentido para o workflow.
- `.claude/rules/` — o que é necessário só em parte do código: convenções de frontend não precisam estar carregadas quando você está mexendo no backend, e vice-versa.

O resultado prático: o arquivo que carrega sempre fica pequeno e focado (o nosso fica em torno de 80 linhas depois de preenchido — bem abaixo do teto de ~200 linhas que a própria Anthropic recomenda), mas o sistema como um todo pode ter uma quantidade grande de conhecimento disponível, porque ele só entra em jogo quando é relevante.

POR QUE UMA ENTREVISTA, E NÃO UM FORMULÁRIO

Você pode estar se perguntando por que o setup é uma conversa guiada pela IA, em vez de um formulário simples que você preenche e envia. A resposta tem a ver com qualidade de resultado, não só conviência.

Um formulário estático assume que você sabe exatamente o que precisa preencher em cada campo, e assume que a informação necessária é sempre a mesma para todo projeto. Na prática, os dois pressupostos falham: um projeto de e-commerce tem regras de negócio completamente diferentes de um projeto de gestão de conteúdo, e o que conta como “informação de segurança crítica” muda dependendo se o sistema lida com pagamentos ou não.

Uma entrevista conduzida pela IA resolve isso de duas formas. Primeiro, ela pode fazer perguntas de acompanhamento — se você mencionar que o projeto lida com assinaturas recorrentes, ela pode perguntar especificamente sobre a política de cancelamento e reembolso, em vez de deixar isso como uma lacuna genérica. Segundo, e mais importante: ela pesquisa ativamente as práticas mais atuais para a stack que você descreveu, em vez de aplicar um template genérico que pode estar desatualizado ou simplesmente não se aplicar à sua combinação específica de linguagem, framework e ferramentas.

O QUE SÃO “SKILLS”, TECNICAMENTE

As 10 skills deste kit seguem um formato chamado Agent Skills — um padrão aberto que a Anthropic publicou em dezembro de 2025 e que, em poucos meses, foi adotado por dezenas de outras ferramentas (OpenAI, Google, Microsoft, entre outras). Antes desse padrão existir, cada

ferramenta de IA tinha seu próprio jeito de fazer a mesma coisa — “comandos” no Claude Code, arquivos de instrução no Copilot, e por aí vai. Skills resolveram isso ao definir um formato único: uma pasta com um arquivo SKILL .md dentro, contendo uma descrição curta (que a IA usa para decidir quando ativar aquela skill) e instruções detalhadas (que só são carregadas quando a skill é de fato usada).

Isso é o mesmo princípio de carregamento sob demanda do CLAUDE .md, aplicado a procedimentos específicos em vez de contexto de projeto. Uma skill de revisão de código não precisa estar “presente” na sua sessão enquanto você está debugando um erro — ela só entra quando você realmente pede uma revisão.

POR QUE UM HOOK, E NÃO SÓ UMA INSTRUÇÃO

Toda a estrutura descrita até aqui — CLAUDE .md, agent_docs/, skills — funciona como contexto: informação que a IA lê e tenta seguir. Isso é poderoso, mas tem um limite real: nenhuma instrução em texto garante 100% de adesão em toda situação. Sob pressão de contexto (uma sessão longa, uma tarefa complexa, muita informação acumulada), até uma instrução clara pode ser negligenciada.

Para a única regra que realmente não pode falhar — validar o código antes de um commit — o kit usa um mecanismo diferente: um hook. Um hook não é uma sugestão que a IA interpreta; é um comando real, que roda automaticamente em um momento específico do ciclo de vida da ferramenta, e que pode bloquear a ação se a validação falhar. É a diferença entre pedir para alguém revisar o próprio trabalho antes de entregar, e ter um sistema que fisicamente não deixa a entrega acontecer se a revisão não passou.

Setup em 5 Minutos

Esse é o workflow mais importante do guia. Ele te leva do zip que você baixou até um projeto com sua IA configurada de verdade — não um template genérico preenchido às pressas, mas um sistema que reflete o seu projeto real, com pesquisa de práticas atuais da sua stack incluída. Cinco minutos. É esse o tempo entre agora e a sua IA parar de perguntar o óbvio.

ANTES DE COMEÇAR

Você precisa de:

- Um projeto (pode ser um projeto já existente ou um novo, vazio)
- Uma ferramenta de IA de código instalada — Claude Code, Cursor, ou qualquer outra que leia arquivos do seu projeto
- 5 minutos de atenção. A entrevista é rápida, mas funciona melhor quando você responde de forma direta, sem pressa de terminar logo

Se você ainda não tem uma ferramenta de IA de código instalada e só usa IA pela web, pule para os 30 prompts contextualizados — eles foram feitos justamente para esse caso, sem precisar da entrevista automática.

PASSO 1 — ONDE COLOCAR OS ARQUIVOS

Dentro do zip do kit, você vai encontrar esta estrutura:

```
CLAUDE.md
agent_docs/
  business-rules.md
  security.md
  engineering-standards.md
  architecture.md
  productivity.md
.claude/
  rules/
    general.md
  hooks/
    README.md
```

Copie essa estrutura inteira para a raiz do seu projeto — a mesma pasta onde fica o `package.json` (se for um projeto Node), o `requirements.txt` (Python), o `go.mod` (Go), ou o arquivo equivalente da sua stack.

Ponto de atenção: a pasta `.claude/` começa com ponto — em alguns sistemas de arquivos, pastas assim ficam ocultas por padrão. Se você copiar pelo Finder (Mac) ou Explorer (Windows) e não ver a pasta `.claude/` no destino, ative a exibição de arquivos ocultos antes de verificar se a cópia funcionou.

PASSO 2 — O COMANDO QUE INICIA TUDO

Abra sua ferramenta de IA dentro do projeto (terminal, dentro da pasta do projeto, se for Claude Code) e cole exatamente este comando:

```
Leia o arquivo CLAUDE.md deste projeto — ele começa com um bloco de comentário HTML com instruções para você ("META-INSTRUCTIONS FOR AI SETUP"). Siga essas instruções: me entreviste sobre este projeto — stack, regras de negócio críticas, segurança, ferramentas que já uso — pesquise as melhores práticas atuais para o stack que eu usar, e distribua o conteúdo entre os arquivos certos exatamente como as instruções descrevem. Configure também o hook de exemplo em .claude/hooks/ com os comandos reais deste projeto. Ao final, remova os blocos de instrução e me mostre o resultado.
```

```
Eu uso: [liste as ferramentas que você usa — ex. "só Claude Code" ou "Claude Code e Cursor"] — se eu usar mais de uma ferramenta, siga a instrução de multi-tool setup (AGENTS.md como fonte da verdade). Se eu usar só Claude Code, preencha o CLAUDE.md direto, sem criar AGENTS.md.
```

Por que colar exatamente esse texto, e não só pedir “configura o CLAUDE.md pra mim”: o comando acima aponta a IA para o bloco de instruções técnicas que já está dentro do arquivo — sem ele, a IA pode preencher os placeholders de forma genérica, sem pesquisar, sem distribuir o conteúdo entre os arquivos certos. É a diferença entre um setup superficial e um setup que realmente funciona como projetado.

PASSO 3 — A ENTREVISTA

A partir daqui, a IA assume a condução. Responda rápido e direto — é isso que mantém os 5 minutos. Ela vai te perguntar, em grupos pequenos de perguntas (não uma lista de 20 de uma vez):

1. O que o projeto faz — descrição curta, o suficiente pra qualquer sessão futura entender o propósito em poucas linhas
2. Stack — linguagem, framework, banco de dados, gerenciador de pacotes, onde é hospedado
3. Convenções existentes — se você já tem um padrão de código, de commit, de estrutura de pastas
4. Regras de negócio críticas — o que o sistema nunca pode fazer errado (ex: “uma assinatura nunca pode ser cancelada sem estornar o valor proporcional”)
5. Segurança — se o projeto lida com pagamentos, dados pessoais, autenticação, e como isso é tratado hoje
6. Ferramentas de produtividade — se você já usa GitHub CLI, Supabase, Linear, ou algo do tipo

Como responder bem:

- Seja específico. “Uso Node” é menos útil que “Node 22, Express, Prisma com PostgreSQL, deploy na Railway.”
- Para regras de negócio, pense em situações reais que já aconteceram ou que você tem certeza que não podem acontecer — não precisa ser exaustivo, só real.
- “Ainda não sei” é uma resposta válida. Um projeto novo, sem regras de negócio definidas ainda, é normal — é melhor a IA marcar isso claramente do que você inventar uma resposta só para preencher.

Durante a entrevista, a IA também vai pesquisar na internet as práticas atuais e a versão recomendada da sua stack — isso é intencional, é parte do que faz este kit diferente de um template estático. Se a sua ferramenta não tiver acesso à internet naquele momento, ela deve

avisar isso explicitamente e usar o conhecimento mais atualizado que tiver, sinalizando o que vale conferir depois.

PASSO 4 — REVISANDO O RESULTADO

Ao final da entrevista, você vai ter um conjunto de arquivos preenchidos com informação real do seu projeto — sem os blocos de instrução de setup (esses são removidos automaticamente, porque não fazem sentido depois que o trabalho deles termina).

Dê uma lida em cada arquivo antes de aceitar como está. Preste atenção especial em:

- `agent_docs/business-rules.md` — geralmente o arquivo mais valioso do sistema. Confirme que as regras estão certas; um erro aqui se propaga para todo o trabalho futuro da IA no projeto.
- `.claude/hooks/README.md` — confirme que os comandos de lint e type-check ali são os comandos reais do seu projeto, não um placeholder esquecido.

Se algo não fizer sentido, é só pedir para a IA ajustar aquele trecho específico — não precisa refazer a entrevista inteira.

PASSO 5 — COMMIT E PRONTO

Suba tudo para o seu repositório: `CLAUDE.md`, `agent_docs/`, `.claude/`, e — se você usa mais de uma ferramenta — o `AGENTS.md` e qualquer arquivo específico daquela ferramenta.

Pronto. A partir de agora, toda sessão nova de IA nesse projeto já nasce sabendo o contexto. Você não precisa mais explicar stack, convenções, ou regras de negócio no começo de cada conversa — os 5 minutos que você acabou de investir se pagam já na próxima vez que abrir uma sessão.

Por que isso importa de verdade, não só na teoria: um estudo de fevereiro de 2026 (Gloaguen et al., ETH Zurich/LogicStar.ai, testando 138 tarefas reais) comparou arquivos de contexto escritos por desenvolvedores com arquivos gerados automaticamente por IA sem entrevista real. O resultado: arquivos gerados de forma genérica, sem esse processo, tiveram desempenho pior do que não ter arquivo de contexto nenhum — reduziram a taxa de sucesso e aumentaram o custo em tokens. Arquivos escritos com informação real e específica do projeto tiveram desempenho consistentemente melhor. É exatamente por isso que a entrevista deste kit não é uma formalidade — pular ela ou responder de forma vaga anula boa parte do valor do sistema.

As 10 Skills no Dia a Dia

COMO UMA SKILL É ATIVADA

As 10 skills deste kit seguem o Agent Skills, um padrão aberto criado pela Anthropic e hoje adotado por mais de 30 ferramentas de IA de código — incluindo Claude Code, Cursor, GitHub Copilot, Gemini CLI e Codex. Isso significa duas coisas práticas:

1. Você não precisa “chamar” uma skill manualmente na maioria dos casos — a IA lê a descrição de cada skill instalada e ativa a que combina com o que você pediu, automaticamente.
2. A mesma pasta de skill funciona sem nenhuma alteração em praticamente qualquer ferramenta de IA que você use — só muda o lugar onde ela fica instalada (veja o Workflow 3 para os detalhes).

Até serem ativadas, as skills custam praticamente nada de contexto — só o nome e a descrição ficam disponíveis “de prontidão.” Isso quer dizer que você pode ter as 10 instaladas sem se preocupar em estar “carregando” o Claude com informação desnecessária.

Uma nota sobre segurança, já que o assunto é relevante: por ser um padrão aberto, existem centenas de milhares de skills públicas disponíveis para instalação livre — e análises recentes do setor (Snyk, SkillsBench) encontraram falhas de segurança reais em cerca de um terço das skills públicas testadas, incluindo casos de payloads maliciosos. Isso não é um problema do padrão em si, é um problema de curadoria: skill é código que a IA executa com base em instruções escritas por outra pessoa, então instalar uma sem revisar tem o mesmo risco que instalar qualquer dependência de código aberto sem checar a fonte. As 10 skills deste kit foram escritas e revisadas especificamente para este produto — mas, se você adicionar skills de terceiros no futuro, vale aplicar o mesmo cuidado que você já aplica a uma dependência nova de npm ou pip.

AS 10 SKILLS, UMA POR UMA

Para cada skill: quando ela ativa, e um exemplo real de como pedir.

Code Review

Ativa quando você pede uma revisão de código, quer que um diff seja checado antes de commitar ou abrir PR, ou pergunta se o código está bom.

`“Revisa esse código antes de eu commitar” · “O que está errado nessa função?”`

Organiza os achados por severidade — bloqueante, deveria corrigir, considerar — e cruza automaticamente com as regras de negócio e segurança documentadas no seu `agent_docs/`.

Debug Assistant

Ativa quando você reporta um bug, cola uma mensagem de erro ou stack trace, ou diz que algo “funcionava antes e agora não.”

`“Esse erro está aparecendo: [cole o erro aqui]” · “Funciona no meu computador mas não em produção”`

Segue um processo estruturado: reproduzir primeiro, depois isolar, depois diagnosticar a causa raiz — não só o sintoma — e sempre com um teste que prova que o bug foi corrigido de verdade.

PR Writer

Ativa quando você pede para abrir um PR ou escrever a descrição de um PR.

`“Escreve a descrição do PR pra essa branch”`

Lê o diff real e o histórico de commits — não escreve a partir do que foi conversado, porque a conversa e o código podem ter se distanciado.

Test Generator

Ativa quando você pede para adicionar testes ou melhorar cobertura.

`“Escreve testes pra essa função” · “Preciso de mais cobertura nesse arquivo”`

Cobre o comportamento esperado, casos de borda reais (não só o caminho feliz), e, se for teste de correção de bug, o cenário exato que costumava falhar.

Doc Writer

Ativa quando você pede para documentar uma função, atualizar um README, ou explicar código para outros desenvolvedores.

`“Documenta essa função” · “Atualiza o README com as novas instruções de setup”`

Só comenta o “porquê”, nunca o “o quê” óbvio — evita aquele tipo de comentário que só repete o que a linha de código já diz.

Frontend Design

Ativa quando você está construindo uma UI nova, uma página, um componente, ou reformulando uma interface existente.

`“Cria uma landing page pra esse produto” · “Preciso de um dashboard de métricas”`

Evita o padrão genérico e templated por padrão — toma decisões deliberadas de tipografia, hierarquia e espaçamento antes de escrever qualquer código, e já inclui acessibilidade e os estados que UI real precisa (carregando, vazio, erro).

Security Audit

Ativa quando você pede uma revisão de segurança, menciona uma vulnerabilidade, está mexendo em autenticação, pagamentos ou dados pessoais, ou pergunta se algo é seguro.

`“Essa rota de pagamento está segura?” · “Faz uma auditoria de segurança nesse módulo de login”`

Não entrega um checklist genérico — pesquisa os riscos reais e atuais da sua stack específica, e explica o caminho de exploração real de cada achado, não só “isso poderia ser um problema.”

API Design

Ativa quando você está criando ou revisando um endpoint de API.

`“Cria um endpoint pra listar os pedidos do usuário” · “Revisa esse endpoint que eu criei”`

Garante consistência com o resto da sua API — mesmo formato de erro, mesmo padrão de nomenclatura — e cobre os fundamentos que costumam ser esquecidos: paginação, status HTTP corretos, validação na fronteira.

Refactor Guide

Ativa quando você pede para refatorar, limpar, simplificar, ou reestruturar código sem mudar o comportamento dele.

`“Essa função ficou grande demais, refatora ela” · “Limpa essa duplicação de código”`

Sempre estabelece uma rede de segurança (testes) antes de mexer em qualquer coisa, e muda em passos pequenos e verificáveis — nunca uma reescrita grande de uma vez.

Feature Planner

Ativa antes de você começar a escrever código para qualquer feature nova, mudança não-trivial, ou tarefa com escopo pouco claro.

“Preciso adicionar um sistema de notificações, me ajuda a planejar” · “Como eu deveria construir isso?”

Essa é a skill que evita o erro mais caro em desenvolvimento com IA: começar a codar antes de entender o problema direito. Ela faz perguntas quando o escopo está ambíguo, escreve um plano curto e verificável, e só avança para o código depois de confirmação — para mudanças que realmente justificam isso.

QUANDO PEDIR EXPLICITAMENTE

Na maioria das vezes, você não precisa mencionar o nome da skill — só descreva o que precisa, do jeito natural que você já fala, e a IA ativa a skill certa sozinha. Mas se notar que uma skill não ativou quando deveria (acontece, principalmente em pedidos muito curtos ou vagos), você pode forçar mencionando diretamente:

```
"Usa a skill de code review nesse diff"
```

Isso é raro de precisar, mas é bom saber que a opção existe.

Antes e Depois

Os resumos do Workflow 2 mostram quando cada skill ativa. Esta seção mostra o que muda na prática, com quatro exemplos mais completos — escolhidos por serem os cenários mais comuns do dia a dia.

CODE REVIEW – REVISÃO GENÉRICA VS. REVISÃO COM CONTEXTO

Sem a skill, pedido genérico:

“Dá uma olhada nesse código aqui.”

Resultado típico: uma lista de observações gerais sobre estilo, nomenclatura, e talvez um ou dois comentários sobre performance — misturando coisas triviais com coisas importantes, sem hierarquia clara do que precisa ser corrigido antes de ir pra produção e do que é só uma sugestão.

Com a skill Code Review ativa, o mesmo pedido gera um resultado estruturado por severidade:

```
### Blocking
- src/payments/refund.ts:34 – o valor do reembolso não está sendo
validado contra o valor original da cobrança, permitindo reembolso
maior que o valor pago – validar refund_amount <= original_amount
antes de processar

### Should fix
- src/payments/refund.ts:51 – erro de rede na chamada ao gateway não
tem retry, então uma falha temporária vira falha permanente para o
usuário

### Consider
- src/payments/refund.ts:12 – a função está com 80 linhas; extrair a
lógica de validação para uma função separada melhoraria a legibilidade
```

A diferença não é só formatação — é que a skill sabe cruzar a mudança com `agent_docs/business-rules.md` e `agent_docs/security.md`, então um problema como “reembolso maior que o valor pago” é identificado como crítico porque viola uma regra de negócio documentada, não porque “parece errado” de forma genérica.

DEBUG ASSISTANT — CORRIGIR O SINTOMA VS. CORRIGIR A CAUSA

Cenário: um erro `TypeError: Cannot read property 'email' of undefined` aparece intermitentemente em produção.

Correção ingênua (o tipo de resposta que uma IA sem processo estruturado tende a dar): adicionar uma verificação `if (user && user.email)` no ponto exato onde o erro aparece. O erro para de acontecer — mas a causa raiz (por que `user` estava `undefined` ali) nunca foi investigada, e é provável que o mesmo problema apareça em outro lugar do código que também assume que `user` sempre existe.

Com a skill Debug Assistant ativa, o processo muda:

1. Pede os passos exatos para reproduzir, ou os logs do momento da falha
2. Escreve um teste que reproduz a condição de corrida
3. Investiga por que `user` chega `undefined` — descobre que é uma condição de corrida entre o carregamento da sessão e a primeira renderização do componente
4. Corrige a causa raiz (garantir que a sessão termine de carregar antes do componente renderizar), não só o sintoma
5. Verifica se o mesmo padrão de acesso não-seguro aparece em outros componentes que dependem do mesmo carregamento de sessão

O resultado é mais lento de produzir na primeira vez, mas evita a situação comum de “corrigir” o mesmo bug em três lugares diferentes do código ao longo de várias semanas.

SECURITY AUDIT — CHECKLIST GENÉRICO VS. RISCO REAL

Pedido: “Essa rota de criação de usuário está segura?”

Checklist genérico (o que uma auditoria superficial produziria): uma lista de boas práticas gerais — “use HTTPS”, “valide o input”, “não exponha stack traces” — sem indicar quais dessas já estão implementadas no código real e quais são lacunas de verdade.

Com a skill **Security Audit ativa**, a resposta é ancorada no código real e no que `agent_docs/security.md` documenta como sensível:

```
### Critical (exploitable now)
- routes/users.ts:22 – a senha é armazenada com um hash de MD5 (linha 22), que é criptograficamente quebrado – qualquer vazamento do banco expõe as senhas de forma prática. Trocar para bcrypt ou argon2id.

### Should address
- routes/users.ts:15 – o endpoint não tem rate limiting, permitindo criação de contas em massa (útil para spam ou enumeração de emails)

### Defense in depth (optional hardening)
- routes/users.ts:8 – considerar adicionar verificação de email antes de ativar a conta, reduzindo contas falsas
```

A diferença central: a skill não lista “boas práticas de segurança em geral” — ela aponta a linha exata, explica por que aquilo é explorável de verdade, e separa o que é urgente do que é reforço opcional.

FEATURE PLANNER – DIRETO PARA O CÓDIGO VS. PLANEJAR PRIMEIRO

Pedido: “Preciso adicionar notificações por email quando um pedido muda de status.”

Direto para o código: a IA já começa a escrever a integração com um serviço de email, assumindo decisões importantes no caminho — qual serviço usar, se as notificações são síncronas ou assíncronas, o que acontece se o envio falhar, se o usuário pode desativar notificações. Se qualquer uma dessas suposições estiver errada, parte do código precisa ser refeita.

Com a skill **Feature Planner ativa**, antes de qualquer código:

Goal

Notificar o usuário por email quando o status de um pedido muda.

Approach

1. Confirmar qual serviço de email o projeto já usa (verificar package.json e variáveis de ambiente) – verificável: encontrar a configuração existente
2. Adicionar um listener no evento de mudança de status que dispara o envio de forma assíncrona (fila), não bloqueando a resposta da API – verificável: mudança de status não fica mais lenta
3. Criar o template de email e o envio – verificável: email chega com as informações corretas em ambiente de teste
4. Adicionar tratamento de falha (retry + log) para envios que falharem – verificável: simular falha do serviço de email e confirmar que o pedido não fica bloqueado

Out of scope

- Preferências de notificação do usuário (ativar/desativar) – feature separada

Open questions / trade-offs

- Envio síncrono seria mais simples de implementar, mas atrasaria a resposta da API se o serviço de email estiver lento. Recomendo assíncrono (fila) – confirma?

Esse plano leva menos de um minuto para a IA escrever, e evita a situação mais cara em desenvolvimento assistido por IA: descobrir só depois de metade do código pronto que a abordagem escolhida não era a certa.

Múltiplas Ferramentas

POR QUE ISSO IMPORTA

Muitos desenvolvedores usam mais de uma ferramenta de IA — Claude Code no terminal para tarefas mais autônomas, Cursor no editor para o dia a dia visual, talvez Copilot pelo hábito antigo. O problema é que, até 2025, cada ferramenta tinha seu próprio formato de configuração: `CLAUDE.md`, `.cursorrules`, `.github/copilot-instructions.md` — e manter os três sincronizados manualmente é um trabalho que ninguém faz por muito tempo. As convenções acabam divergindo, e uma ferramenta passa a dar respostas diferentes da outra para o mesmo projeto.

Isso mudou: o mercado convergiu em um arquivo aberto chamado `AGENTS.md`, hoje lido nativamente por Cursor, GitHub Copilot, Gemini CLI, Windsurf (agora Devin Desktop) e a maioria das outras ferramentas. O Claude Code é a exceção — ele só lê `CLAUDE.md` — mas o sistema deste kit já resolve isso para você.

COMO O KIT LIDA COM ISSO

Quando você contou pra IA, no Passo 2 do Workflow 1, quais ferramentas usa, o comando de setup já seguiu automaticamente a estratégia certa:

- **Se você usa só Claude Code:** o `CLAUDE.md` foi preenchido diretamente, sem complicação extra.
- **Se você usa mais de uma ferramenta:** o `AGENTS.md` guarda todo o conteúdo real — é a fonte única da verdade. O `CLAUDE.md` virou um “atalho”: só uma linha (`@AGENTS.md`) que importa o conteúdo de lá, mais qualquer instrução específica só do Claude Code, se houver.

Isso significa que, se você mudar uma convenção do projeto no futuro, muda em um lugar só — o `AGENTS.md` — e todas as ferramentas que você usa continuam enxergando a mesma informação, sem risco de divergência.

ONDE CADA FERRAMENTA GUARDA A CONFIGURAÇÃO

FERRAMENTA	LÊ NATIVAMENTE	ONDE FICA
Claude Code	Só CLAUDE.md	raiz do projeto
Cursor	AGENTS.md + .cursor/rules/*.mdc (regras com escopo)	raiz + .cursor/rules/
GitHub Copilot	AGENTS.md + .github/copilot-instructions.md (os dois juntos, se existirem)	raiz + .github/
Windsurf / Devin Desktop	AGENTS.md + .devin/rules/	raiz + .devin/rules/
Codex, Gemini CLI, outras	AGENTS.md (padrão aberto)	raiz do projeto

ADICIONANDO UMA FERRAMENTA DEPOIS

Se você começou só com Claude Code e mais tarde passou a usar Cursor também, não precisa refazer a entrevista inteira. É só pedir:

```
Hoje eu só tenho CLAUDE.md configurado neste projeto. Agora também uso  
[Cursor / Copilot / outra]. Siga a instrução de multi-tool setup: crie  
o AGENTS.md com o conteúdo que já está no CLAUDE.md, transforme o  
CLAUDE.md num wrapper fino que importa o AGENTS.md, e crie o arquivo de  
escopo específico dessa ferramenta se ela tiver um.
```

O mesmo vale para as 10 skills: elas seguem um padrão aberto separado (Agent Skills), então basta copiar a mesma pasta para o diretório de skills da nova ferramenta — sem reescrever nada. Isso está detalhado no arquivo COMO-INSTALAR.md dentro da pasta de skills.

Mantendo o Sistema Vivo

O SISTEMA APRENDE COM VOCÊ

Um dos erros mais comuns com esse tipo de configuração é tratá-la como um artefato de instalação único — configura uma vez, esquece, e seis meses depois ela está desatualizada e a IA está seguindo regras que não existem mais.

Este kit foi desenhado para não sofrer disso. Dentro do CLAUDE.md (ou do AGENTS.md, se você usa multi-ferramenta), existe uma seção chamada “Keeping This File Current” — ela instrui a IA a, durante o trabalho normal no projeto, avisar quando perceber algo que deveria virar regra permanente: uma regra de negócio nova que surgiu na conversa, uma decisão de arquitetura, um “nunca faça X aqui” que você mencionou de passagem.

Quando isso acontecer, a IA vai perguntar se deve documentar aquilo — e em qual arquivo. Aceite essas sugestões quando fizerem sentido. É esse hábito, mais do que qualquer configuração inicial, que faz o sistema continuar útil daqui a um ano.

O HOOK DE VALIDAÇÃO

Dentro de `.claude/hooks/`, existe um exemplo funcional de hook — um mecanismo diferente de tudo mais no kit, porque ele não é uma sugestão que a IA pode ignorar sob pressão de contexto. É um comando real que roda automaticamente antes de qualquer commit, executa lint e type-check, e bloqueia o commit de verdade se algo falhar.

Isso é intencional: toda outra regra deste sistema é contexto — a IA lê e tenta seguir, mas nada garante 100% de adesão em toda situação. O hook é a única parte do kit com garantia mecânica. Se você configurou ele durante a entrevista com os comandos reais do seu projeto (e não deixou como placeholder), ele já está funcionando.

QUANDO E COMO ATUALIZAR

Além do fluxo automático de sugestões, alguns momentos valem uma atualização manual:

- O projeto trocou de stack ou adicionou uma dependência grande — peça para a IA atualizar CLAUDE.md / AGENTS.md e os arquivos de `agent_docs/` relevantes.

- Uma nova pessoa entrou no time e trouxe convenções diferentes — documente a decisão final, não as duas opções.
- Você percebeu que a IA cometeu o mesmo erro duas vezes — isso é sinal de que falta uma regra explícita em algum lugar. Adicione-a.
- Passaram alguns meses desde o setup inicial — vale pedir para a IA revisar se as recomendações de ferramentas e versões em `agent_docs/engineering-standards.md` ainda são as atuais, já que tecnologia muda rápido.

Pedir um ajuste pontual é sempre mais rápido do que deixar o sistema acumular desatualização e precisar de uma reescrita completa depois.

Por Tipo de Projeto

O sistema se adapta ao seu projeto durante a entrevista, mas alguns padrões se repetem dependendo do tipo de trabalho. Esta seção mostra como o kit costuma se comportar em cada cenário — para você saber o que esperar antes mesmo de começar.

PROJETO NOVO, DO ZERO

Se você está começando um projeto do zero, boa parte da entrevista terá respostas do tipo “ainda não sei” ou “ainda não existe” — e isso é esperado, não um problema. `agent_docs/business-rules.md` pode começar quase vazio, com só uma nota de que regras serão adicionadas conforme surgirem. O que mais importa nesse cenário é o `CLAUDE.md` e `agent_docs/engineering-standards.md`: definir desde o início como o código deve ser estruturado, testado e documentado economiza retrabalho mais tarde, quando o projeto já tiver muito código escrito sob padrões inconsistentes.

Nesse cenário, a skill Feature Planner tende a ser a mais usada nas primeiras semanas — cada decisão estrutural inicial (como organizar pastas, que padrão de API seguir, como estruturar o banco de dados) se beneficia de um plano curto antes da implementação.

PROJETO EXISTENTE, SEM DOCUMENTAÇÃO

Este é provavelmente o cenário mais comum: um projeto que já roda em produção, com convenções que existem na prática mas nunca foram escritas em lugar nenhum — o time “simplesmente sabe” como as coisas são feitas.

Aqui, a entrevista funciona melhor se você tiver o código aberto ao lado enquanto responde. Em vez de tentar lembrar de memória todas as convenções, você pode pedir para a própria IA ler o código existente primeiro e propor um rascunho do `CLAUDE.md` baseado no que encontrar, que você depois corrige e confirma. O comando muda ligeiramente:

“Antes de me entrevistar, dê uma olhada nos arquivos principais do projeto para entender os padrões que já existem. Depois me pergunte só o que não conseguir inferir do código — decisões de negócio, coisas que não estão explícitas no código, e o que você percebeu que parece inconsistente.”

Isso costuma revelar inconsistências reais — dois padrões diferentes de nomenclatura convivendo no mesmo projeto, por exemplo — que vale a pena resolver (ou pelo menos documentar qual é o padrão “oficial” daqui para frente) durante o próprio setup.

PROJETO COM MÚLTIPLOS DESENVOLVEDORES

Se você não é a única pessoa trabalhando no projeto, dois pontos do sistema merecem atenção extra. Primeiro, `agent_docs/business-rules.md` vale uma revisão em grupo, não só individual — regras de negócio às vezes vivem só na cabeça de quem lida mais de perto com aquela parte do sistema, e o processo de escrever elas explicitamente é uma boa oportunidade para alinhar o time, não só para configurar a IA.

Segundo, a seção “Keeping This File Current” (Workflow 4) fica ainda mais importante em equipe: se cada pessoa aceita as sugestões de atualização da IA de forma independente, o sistema evolui de forma consistente. Se só uma pessoa mantém o arquivo atualizado, ele rapidamente reflete só o conhecimento dela, não o do time todo.

API OU BACKEND PURO, SEM INTERFACE

Projetos que são só API — sem frontend — tendem a usar intensamente as skills API Design, Security Audit e Test Generator, e quase nunca a Frontend Design (o que é esperado — não custa nada ter a skill instalada mesmo sem usar, já que ela só carrega quando ativada).

Vale prestar atenção especial ao preenchimento de `agent_docs/security.md` nesse cenário: uma API é, por definição, uma superfície de ataque direta, então documentar claramente o modelo de autenticação e quais rotas lidam com dados sensíveis compensa o tempo investido.

APLICAÇÃO COM PAGAMENTOS OU DADOS SENSÍVEIS

Se o projeto lida com pagamentos, dados de saúde, ou qualquer informação regulada, vale ser especialmente detalhado nas seções de regras de negócio e segurança durante a entrevista — esses são exatamente os dois arquivos que a skill Code Review cruza automaticamente com toda mudança de código, e quanto mais completos estiverem, mais a revisão automática pega antes de chegar em produção.

Um padrão que vale adotar nesse cenário: depois do setup inicial, peça explicitamente para a skill Security Audit rodar uma vez sobre o projeto inteiro (não só sobre uma mudança pontual), para ter um retrato do estado atual antes de seguir desenvolvendo.

MONOREPO OU MÚLTIPLOS SERVIÇOS

Para projetos com mais de um serviço no mesmo repositório, o `agent_docs/architecture.md` passa a ser um dos arquivos mais importantes — é onde ficam documentados os limites entre serviços, o que cada um é responsável por fazer, e por que a divisão é daquele jeito (o que evita que uma sessão futura de IA “conserte” uma separação que era intencional).

Vale também considerar `.claude/rules/` com escopo por pasta desde o início — uma regra que só se aplica ao serviço de pagamentos não precisa carregar quando a tarefa é no serviço de notificações.

Organizado por Sintoma

Esta seção está organizada por sintoma — o que você está vendo de errado — não por funcionalidade. Procure pela situação mais parecida com a sua.

“A IA não fez a entrevista, só preencheu os placeholders com coisas genéricas”

Isso costuma acontecer quando o comando de setup foi resumido ou reescrito à mão em vez de colado exatamente como está no Passo 2 do Workflow 1. O comando exato é o que garante que a IA vá ler o bloco de meta-instruções dentro do CLAUDE.md antes de agir. Cole o comando completo novamente, sem alterações, e peça para ela recomençar o processo.

“Uma das skills não está ativando quando eu peço algo que deveria disparar ela”

Primeiro, confirme que a pasta da skill está no lugar certo — veja a tabela no COMO-INSTALAR.md dentro da pasta de skills para o caminho exato da sua ferramenta. Se o caminho está certo e ainda assim não ativa, tente mencionar a skill diretamente (veja o final do Workflow 2) — pedidos muito curtos ou ambíguos às vezes não dão contexto suficiente para a ativação automática funcionar.

“O hook não está bloqueando o commit mesmo com o lint falhando”

Confira se `.claude/settings.json` foi realmente criado durante a entrevista, e se os comandos dentro dele (`[LINT_COMMAND]` e `[TYPECHECK_COMMAND]`) foram substituídos pelos comandos reais do seu projeto — se ficaram como placeholder, o hook não tem o que executar. Veja o `.claude/hooks/README.md` para o formato exato e um exemplo já preenchido.

“Eu uso Cursor (ou outra ferramenta) e ela não está lendo o CLAUDE.md”

Isso é esperado — cada ferramenta lê um arquivo diferente por padrão. Se você contou pra IA, no Passo 2 do Workflow 1, que usa mais de uma ferramenta, o setup já deveria ter criado um `AGENTS.md` na raiz do projeto, que é o arquivo que o Cursor (e a maioria das outras ferramentas) lê nativamente. Confira se esse arquivo existe. Se não existir, veja “Adicionando uma ferramenta depois” no Workflow 3 para o comando que resolve isso sem refazer a entrevista.

“As recomendações de ferramentas (lint, testes) que a IA sugeriu não fazem sentido pra minha linguagem”

As ferramentas de qualidade citadas no sistema (lint, type-check, testes, hook de pré-commit) são conceitos, não uma lista fixa — a implementação real varia por linguagem. Se a IA sugeriu algo que não existe ou não é padrão na sua stack, isso normalmente significa que ela não pesquisou a fundo durante a entrevista (às vezes acontece quando a sessão não tinha acesso à internet naquele momento). Peça explicitamente: “pesquise as ferramentas de lint e teste padrão para [sua linguagem] e atualize o CLAUDE.md com o que for real.”

“Uma regra que eu documentei em agent_docs/business-rules.md não está sendo respeitada pela IA em uma sessão nova”

Confirme que o arquivo está sendo referenciado corretamente no CLAUDE.md (procure pela seção “Deeper Context”) — se o link estiver quebrado ou o nome do arquivo tiver sido alterado manualmente, a IA não consegue carregá-lo sob demanda. Se o link estiver correto e o problema persistir, tente ser mais explícito na própria sessão: “antes de continuar, leia agent_docs/business-rules.md.”

“Já se passaram alguns meses e eu não sei se as recomendações ainda são atuais”

Isso é normal e esperado — tecnologia muda. Veja “Quando e como atualizar” no Workflow 4. A forma mais simples é pedir: “revise agent_docs/engineering-standards.md e confirme se as ferramentas e versões recomendadas ainda são as atuais para este stack, pesquisando antes de responder.”

Dúvidas Sobre o Sistema

Esta seção reúne perguntas que não se encaixam exatamente no Troubleshooting (que é sobre “algo deu errado”) — são dúvidas legítimas sobre como o sistema funciona ou decisões de uso.

Preciso saber programar para usar isso?

Sim, no sentido de que este é um kit para quem já desenvolve software — os arquivos, as regras de negócio, e as skills assumem que você entende conceitos como stack, endpoint, teste, e commit. Não é um curso que ensina a programar; é infraestrutura para quem já programa e quer que a IA colabore melhor no processo.

O kit funciona para qualquer linguagem de programação?

Sim — esse é um dos pontos centrais do design. Nenhum arquivo do sistema (CLAUDE.md, os agent_docs/, as skills) assume uma linguagem específica. As recomendações técnicas (ferramentas de lint, testes, etc.) são preenchidas durante a entrevista com o que é real e atual para a sua stack específica — o mesmo kit serve para um projeto em TypeScript, Python, Go, Rust, ou qualquer outra linguagem.

Preciso rodar a entrevista de novo se eu mudar de computador?

Não. O sistema inteiro vive dentro do repositório do projeto (é por isso que o Passo 5 do Workflow 1 pede para subir tudo com o commit). Se você clonar o repositório em outro computador, os arquivos já estão lá — não é uma configuração local da sua máquina.

E se eu tiver mais de um projeto? Preciso comprar o kit de novo?

Não — você instala o mesmo kit em quantos projetos quiser. Cada projeto terá sua própria entrevista e seu próprio conjunto de arquivos preenchidos (já que cada projeto tem stack, regras de negócio e convenções diferentes), mas o material-fonte (o CLAUDE.md template, as 10 skills) é reutilizado à vontade.

As skills funcionam mesmo sem o CLAUDE.md configurado?

Tecnicamente sim — skills e o sistema de CLAUDE.md / agent_docs/ são mecanismos independentes. Mas na prática, elas funcionam melhor juntas: várias skills (Code Review, Security Audit, Feature Planner) foram desenhadas para cruzar informação com agent_docs/business-rules.md e agent_docs/security.md. Sem esses arquivos preenchidos, as skills ainda funcionam, só que com menos contexto do seu projeto específico. Recomendamos sempre fazer o Workflow 1 completo antes de depender pesadamente das skills.

Posso editar o conteúdo dos arquivos manualmente, sem passar pela IA?

Sim, completamente. Depois que os arquivos estão preenchidos, eles são só markdown — você pode abrir e editar como qualquer outro arquivo de texto do projeto. A entrevista guiada é a forma recomendada de preenchimento inicial (porque pesquisa e organiza automaticamente), mas uma vez preenchidos, edições pontuais diretas são perfeitamente normais — aliás, é exatamente isso que o Workflow 4 recomenda para ajustes pequenos.

O que acontece se eu não seguir nenhuma das sugestões de atualização da IA?

Nada quebra imediatamente — o sistema continua funcionando com a configuração que já existe. O risco é gradual: com o tempo, o CLAUDE.md e os agent_docs/ vão ficando cada vez mais distantes da realidade atual do projeto, e a IA passa a operar com um contexto cada vez menos preciso, sem que haja um “aviso” claro de que isso está acontecendo. É por isso que o Workflow 4 recomenda aceitar as sugestões quando fizerem sentido — é o que mantém o sistema útil ao longo do tempo, em vez de virar um artefato de instalação esquecido.

As 10 skills cobrem tudo que eu preciso, ou vou precisar escrever as minhas próprias?

As 10 skills cobrem o núcleo do trabalho diário de desenvolvimento — revisão, debug, testes, documentação, design de UI, segurança, design de API, refatoração e planejamento. Elas não tentam cobrir todo workflow imaginável (deploy, gestão de projeto, integração com ferramentas específicas de terceiros). Se você perceber que está repetindo o mesmo tipo de instrução detalhada para a IA em toda sessão — algo que nenhuma das 10 skills cobre — esse é exatamente o sinal de que vale escrever uma skill própria para aquele workflow específico, seguindo o mesmo formato (pasta com um arquivo SKILL.md dentro).

O hook de validação impede qualquer commit até tudo estar perfeito?

Não exatamente — ele bloqueia o commit se os comandos de lint e type-check configurados falharem, o que normalmente significa erros reais de sintaxe, tipo, ou estilo que o seu projeto já define como obrigatórios. Ele não julga qualidade de código de forma subjetiva, nem impede commits incompletos ou “trabalho em progresso” — só impede que código que quebra as próprias regras do projeto seja commitado sem querer.

Isso substitui a revisão de código feita por outra pessoa do time?

Não deveria. A skill Code Review é uma primeira camada de revisão — rápida, disponível a qualquer momento, e boa em pegar problemas óbvios antes mesmo de abrir um PR. Mas revisão humana continua tendo valor que a IA não substitui: julgamento sobre trade-offs de produto, conhecimento tácito do time, e a responsabilidade final por decisões importantes. Pense na skill como o que reduz o número de idas e vindas triviais numa revisão humana, não como uma substituição dela.

O que eu faço se a IA sugerir algo que contradiz uma convenção que o time já usa?

Isso normalmente significa que a convenção real do time não está refletida em CLAUDE.md ou em `.claude/rules/` — a IA está seguindo o que está escrito, então se o escrito estiver desatualizado ou incompleto, o resultado também será. A correção é atualizar o arquivo correspondente com a convenção real (veja o Workflow 4), não só corrigir a sugestão pontualmente toda vez que ela aparecer.

Todos os Arquivos do Kit

Todos os arquivos do kit, em uma página.

ARQUIVO	O QUE TEM DENTRO
CLAUDE.md	Núcleo do sistema — contexto do projeto, comandos, ponteiros para os outros arquivos
AGENTS.md (só se multi-ferramenta)	Fonte real de todo o conteúdo, quando você usa mais de uma ferramenta de IA
agent_docs/business-rules.md	Regras de negócio que o projeto nunca pode violar
agent_docs/security.md	Práticas de segurança específicas do seu stack e domínio
agent_docs/engineering-standards.md	Filosofia de qualidade — TDD, performance, documentação
agent_docs/architecture.md	Decisões estruturais do projeto (só se o projeto for complexo o bastante)
agent_docs/productivity.md	Ferramentas de produtividade específicas do seu workflow
.claude/rules/general.md	Convenções gerais + exemplo de regra com escopo por pasta
.claude/hooks/README.md	O hook de validação pré-commit, com exemplo já preenchido
skills/*/SKILL.md (×10)	As 10 skills, uma pasta cada

COMANDOS QUE VOCÊ VAI USAR MAIS

Setup inicial (Workflow 1, Passo 2) — o comando completo está lá, não resumido aqui de propósito, porque precisa ser colado exatamente como está.

Adicionar ferramenta nova (Workflow 3):

```
Hoje eu só tenho CLAUDE.md configurado neste projeto. Agora também uso [ferramenta]. Siga a instrução de multi-tool setup...
```

Atualizar recomendações desatualizadas (Troubleshooting):

Pesquise as ferramentas de lint e teste padrão para [linguagem] e atualize o CLAUDE.md com o que for real.

Forçar uma skill específica (Workflow 2):

Usa a skill de [nome da skill] em [o que você quer analisar]

O QUE ESTE KIT NÃO É

Vale repetir, porque isso molda como usar tudo isso bem: este não é um curso, não tem videoaula, e não é uma ferramenta com interface própria. É infraestrutura — arquivos que você instala uma vez, mantém com o sistema de sugestões automáticas, e que fazem qualquer sessão de IA nesse projeto, daqui em diante, começar sabendo o que precisa saber.

Fim do guia. Bom trabalho — e bons commits.